

172 questions du professeur

Réponse courte, réponse développée, erreur à éviter

Romain · Yannis · Lisa

github.com/RomainJazzar/travelling-merchant

Banque de questions du professeur — 172 questions

Projet « Le marchand ambulant » · Romain · Yannis · Lisa

Mode d'emploi

Chaque question suit le même format :

Réponse courte — ce que vous dites en 1 ou 2 phrases. **Réponse développée** — ce que vous ajoutez si on creuse. ⚠ **Erreur à éviter** — le piège classique.

Niveaux :

Nive au	Nature	Qui doit savoir répondre
1	Questions simples, indispensables	Romain, Yannis, Lisa — tous
2	Compréhension réelle	tous
3	Questions techniques sur les algorithmes	tous, en priorité le responsable de la partie
3 bis	Questions sur le code (GitHub ouvert)	tous
4	Questions pièges	tous
5	Questions très exigeantes	idéalement au moins un du groupe

Rappel des chiffres : Christofides 3 445,60 km · GA 3 157,13 km · MST 2 665,05 km · couplage 1 142,03 km · 12 sommets impairs · écart 288,48 km (8,37 %) · 190 arêtes · config retenue 160/520/5/95 %/22 %/6 · seeds 11, 22, 33.

NIVEAU 1 — Les indispensables

Q1. Quel est le problème que vous résolvez ?

Courte. Le problème du voyageur de commerce : trouver le cycle le plus court qui visite 20 villes une seule fois chacune et revient au point de départ. **Développée.** Notre marchand, Théobald, doit écouler ses marchandises sur 20 marchés français. Chaque détour lui coûte du temps, de l'argent et l'expose à des risques. On traduit ça en : minimiser la distance totale d'un circuit fermé passant une fois par chaque ville. ⚠ Ne dites pas « trouver le plus court chemin » : ce serait Dijkstra. On cherche un **cycle** qui passe par **toutes** les villes.

Q2. Qu'est-ce que le TSP ?

Courte. Travelling Salesman Problem : étant donné n villes et les distances entre elles, trouver le cycle hamiltonien de poids minimal. **Développée.** C'est un des problèmes les plus étudiés de l'optimisation combinatoire, formalisé dans les années 1930. Il modélise la logistique, mais aussi le perçage de cartes électroniques ou le séquençage ADN.

Q3. Combien de villes ?

Courte. 20, listées dans `data/villes_france_lat_long.csv` avec leurs coordonnées GPS. **Développée.** Paris, Marseille, Lyon, Toulouse, Nice, Nantes, Strasbourg, Montpellier, Bordeaux, Lille, Rennes, Reims, Le Havre, Saint-Étienne, Toulon, Grenoble, Dijon, Angers, Nîmes, Clermont-Ferrand.

Q4. Pourquoi ne pas tester toutes les tournées ?

Courte. Avec le départ fixé, il y a $19! \approx 1,2 \times 10^{17}$ tournées. À un milliard par seconde, il faudrait près de 4 ans. **Développée.** Et en ajoutant une seule ville, on multiplie ce temps par 20. Le sujet le dit explicitement : « sans tous les évaluer ». ⚠ Ne dites pas « c'est impossible » : c'est **infaisable en temps raisonnable**, nuance.

Q5. Pourquoi $(n-1)!$ et pas $n!$?

Courte. Parce que dans un cycle, le point de départ n'a pas d'importance : les n écritures d'un même cycle sont la même tournée. On divise $n!$ par n . **Développée.** `Paris→Lyon→Nice→Paris` et `Lyon→Nice→Paris→Lyon` sont le même circuit. Si on tient compte aussi du sens de parcours, on descend même à $(n-1)!/2$, soit $6,08 \times 10^{16}$. ⚠ Ne dites pas $20!$.

Q6. Comment avez-vous représenté le problème ?

Courte. Par un graphe complet pondéré non orienté : 20 sommets, 190 arêtes, poids = distance de Haversine. **Développée.** Chaque ville est un sommet identifié par un entier de 0 à 19 ; les coordonnées sont stockées en attributs de nœud ; les distances sont précalculées dans une matrice 20×20 .

Q7. Qu'est-ce qu'un graphe ?

Courte. Un ensemble de sommets et un ensemble d'arêtes reliant des paires de sommets.

Développée. Formellement $G = (V, E)$. Ici V = les 20 villes, E = les 190 liaisons.

Q8. Qu'est-ce qu'un graphe complet ?

Courte. Un graphe où toute paire de sommets distincts est reliée par une arête. **Développée.** Noté K_n . Chez nous K_{20} . C'est le cas parce qu'on peut calculer Haversine entre n'importe quelles deux villes.

Q9. Combien d'arêtes avec 20 sommets ? Pourquoi 190 ?

Courte. 190, car $C(20,2) = 20 \times 19 / 2$. **Développée.** $20 \times 19 = 380$ compte les **couples ordonnés** (A vers B et B vers A). Comme le graphe est non orienté, l'arête $\{A,B\}$ est unique : on divise par 2. ⚠ 380 est la réponse pour un graphe **orienté**. Ne la donnez pas ici.

Q10. Pourquoi non orienté ?

Courte. Parce que Haversine est symétrique : $d(A,B) = d(B,A)$. **Développée.** Dans le code, on utilise `nx.Graph` et non `nx.DiGraph`, et la matrice est symétrique par construction (on remplit `matrix[i][j]` et `matrix[j][i]` en même temps).

Q11. Pourquoi pondéré ?

Courte. Parce qu'on minimise une distance : sans poids sur les arêtes, « le plus court » n'a aucun sens.

Q12. Qu'est-ce que Haversine ?

Courte. Une formule qui donne la distance de grand cercle entre deux points repérés par latitude et longitude sur une sphère. **Développée.** $d = 2R \cdot \arcsin(\sqrt{h})$ avec $h = \sin^2(\Delta\phi/2) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2(\Delta\lambda/2)$ et $R = 6\,371,0088$ km.

Q13. Pourquoi Haversine et pas la distance euclidienne ?

Courte. Parce que nos coordonnées sont des angles sur une sphère, pas des coordonnées cartésiennes dans un plan. **Développée.** Un degré de latitude vaut toujours ~ 111 km ; un degré de longitude vaut ~ 111 km à l'équateur mais seulement ~ 73 km à la latitude de Paris. Appliquer Pythagore reviendrait à

additionner des unités incomparables.

Q14. Quelle est la distance trouvée par Christofides ?

Courte. 3 445,60 km.

Q15. Quelle est la distance trouvée par l'algorithme génétique ?

Courte. 3 157,13 km — c'est la meilleure tournée que nous ayons observée. $\triangle$ Jamais « c'est l'optimum ».

Q16. Quel est l'écart entre les deux ?

Courte. 288,48 km, soit 8,37 % en faveur du génétique.

Q17. Quelle méthode recommandez-vous ?

Courte. Le génétique pour tracer la route, Christofides comme référence et filet de sécurité.

Développée. Le génétique gagne 288 km sur une tournée que Théobald répète. Mais Christofides s'exécute en 1,5 ms, est déterministe et théoriquement borné : c'est lui qu'on choisit si le temps de calcul devient critique, ou si l'instance grandit.

Q18. Combien de temps prend chaque algorithme ?

Courte. Environ 1,5 ms pour Christofides et environ 1,2 s par exécution pour le génétique, sur notre machine. $\triangle$ Précisez toujours « sur notre machine » : nous avons nous-mêmes observé un facteur 4 entre deux machines sur le GA. Les **distances**, elles, sont identiques.

Q19. Où sont vos résultats ?

Courte. Dans `results/` : `summary.json`, `benchmark_ga.csv`, deux cartes HTML et plusieurs PNG. Tout est régénéré par `python main.py`.

Q20. Votre code est-il public ?

Courte. Oui, sur github.com/RomainJazzar/travelling-merchant, avec un README présentant le contexte, les deux algorithmes et la conclusion.

NIVEAU 2 — Compréhension réelle

Q21. Qu'est-ce qu'une métrique ?

Courte. Une fonction de distance qui vérifie quatre propriétés : non-négativité, identité des indiscernables, symétrie et inégalité triangulaire. **Développée.** Haversine les vérifie toutes les quatre, c'est la distance géodésique sur une sphère.

Q22. Pourquoi l'inégalité triangulaire est-elle si importante ?

Courte. Parce que c'est elle qui rend l'étape de raccourci de Christofides gratuite, et donc qui permet de prouver la garantie de 1,5. **Développée.** Sauter une ville déjà visitée revient à remplacer $A \rightarrow B \rightarrow C$ par $A \rightarrow C$. Sans $d(A, C) \leq d(A, B) + d(B, C)$, ce remplacement pourrait allonger la tournée et toute la preuve s'effondrerait. On parle alors de **TSP métrique**.

Q23. Avez-vous vérifié que l'inégalité est respectée ?

Courte. Oui : nous avons testé les 6 840 triplets de notre matrice, **zéro violation**. **Développée.** C'est donc un fait vérifié, pas une supposition. Nous avons aussi vérifié la symétrie de la matrice et la nullité de sa diagonale.

Q24. Qu'est-ce qu'un arbre ?

Courte. Un graphe connexe sans cycle. Avec n sommets, il a exactement $n-1$ arêtes.

Q25. Qu'est-ce qu'un arbre couvrant ?

Courte. Un arbre qui touche tous les sommets du graphe.

Q26. Qu'est-ce qu'un MST ?

Courte. L'arbre couvrant dont la somme des poids est minimale. Chez nous : 2 665,05 km sur 19 arêtes.

Q27. Pourquoi $MST \leq OPT$?

Courte. Parce qu'en retirant une arête à la tournée optimale, on obtient un arbre couvrant, dont le poids est donc $\geq$ celui du MST. **Développée.** La tournée optimale a n arêtes et touche tous les sommets. Enlevez-en une : vous avez un chemin couvrant, qui est un arbre couvrant particulier. Son poids est $\geq$ MST par définition du minimum, et $\leq$ OPT puisqu'on a retiré une arête. Donc $MST \leq OPT$. $\triangle$ Ne dites pas « le MST est une tournée ».

Q28. Pourquoi le MST n'est-il pas une tournée ?

Courte. Parce qu'un arbre n'a pas de cycle, qu'il a des feuilles de degré 1 et des embranchements de degré ≥ 3 . Dans une tournée, chaque ville a exactement deux voisins.

Q29. Qu'est-ce que le degré d'un sommet ?

Courte. Son nombre d'arêtes incidentes.

Q30. Pourquoi vous intéressez-vous aux sommets de degré impair ?

Courte. Parce qu'un sommet de degré impair empêche l'existence d'un circuit eulérien. **Développée.** Pour traverser un sommet, il faut y entrer par une arête et en ressortir par une autre : les arêtes se consomment par paires. Un degré impair laisse toujours une arête orpheline.

Q31. Combien de sommets impairs dans votre MST ?

Courte. 12 : Lyon, Nice, Strasbourg, Bordeaux, Lille, Reims, Saint-Étienne, Grenoble, Dijon, Angers, Nîmes et Clermont-Ferrand.

Q32. Pourquoi sont-ils forcément en nombre pair ?

Courte. Par le lemme des poignées de main : la somme des degrés vaut $2|E|$, donc elle est paire, et une somme paire ne peut pas contenir un nombre impair de termes impairs. **Développée.** Chaque arête a deux extrémités et contribue 1 au degré de chacune : elle est comptée exactement deux fois. Ce n'est pas une coïncidence de notre instance, c'est un théorème valable pour tout graphe.

Q33. Qu'est-ce qu'un couplage ?

Courte. Un ensemble d'arêtes sans sommet commun.

Q34. Qu'est-ce qu'un couplage parfait ?

Courte. Un couplage qui couvre tous les sommets : chacun appartient à exactement une arête. Il n'existe que si le nombre de sommets est pair.

Q35. Pourquoi « de poids minimal » ?

Courte. Deux raisons : ces arêtes s'ajoutent au coût final, et c'est la minimalité qui permet de prouver que le couplage coûte au plus $OPT/2$. $\triangle$ Ne donnez pas que la raison pratique : la raison théorique est celle qui compte pour la garantie.

Q36. Quel est le coût de votre couplage ?

Courte. 1 142,03 km, sur 6 arêtes. **Développée.** Lyon–Grenoble 94,33 ; Saint-Étienne–Clermont-Ferrand 107,88 ; Lille–Reims 167,62 ; Nice–Nîmes 233,41 ; Strasbourg–Dijon 245,17 ; Bordeaux–Angers 293,62.

Q37. Qu'est-ce qu'un multigraphe ?

Courte. Un graphe qui autorise plusieurs arêtes entre la même paire de sommets. **Développée.** C'est ce qu'on obtient en unissant MST et couplage : une arête du couplage peut déjà exister dans le MST. Notre `_hierholzer_multigraph` gère ça par des identifiants d'arête plutôt que par des paires de sommets.

Q38. Euler vs Hamilton, quelle différence ?

Courte. Euler passe une fois par chaque **arête**, Hamilton une fois par chaque **sommet**. **Développée.** Euler a un critère simple (tous les degrés pairs) et un algorithme en $O(E)$. Hamilton n'a aucun critère simple : décider s'il existe est NP-complet.

Q39. Que cherche le TSP, Euler ou Hamilton ?

Courte. Un cycle **hamiltonien** de poids minimal.

Q40. Pourquoi Christofides construit-il un circuit eulérien alors ?

Courte. Parce qu'un circuit eulérien est facile à construire et qu'on peut ensuite le convertir en cycle hamiltonien sans coût supplémentaire, grâce à l'inégalité triangulaire.

Q41. Qu'est-ce que Hierholzer ?

Courte. L'algorithme qui construit un circuit eulérien en $O(E)$: on avance tant qu'on peut, et quand on est bloqué on ferme un sous-circuit et on remonte.

Q42. Qu'est-ce que le shortcutting ?

Courte. L'étape qui parcourt le circuit eulérien et saute toute ville déjà visitée, pour obtenir un cycle hamiltonien.

Q43. D'où vient le facteur 1,5 ?

Courte. De $MST \leq OPT$ et $\text{couplage} \leq OPT/2$, et du fait que le raccourci n'augmente jamais le coût. Donc $\text{Christofides} \leq MST + \text{couplage} \leq 1,5 \times OPT$. **Développée.** La deuxième borne est plus subtile : en parcourant la tournée optimale et en ne retenant que les sommets impairs, on obtient un cycle sur ces sommets de coût $\leq OPT$ (par inégalité triangulaire). Ce cycle a un nombre pair de sommets, donc il se décompose en deux couplages parfaits disjoints. Le moins cher des deux coûte $\leq OPT/2$. Comme notre couplage est **minimal**, il est au moins aussi bon.

Q44. Christofides donne-t-il l'optimum ?

Courte. Non. C'est un algorithme d'**approximation** : il garantit de ne jamais dépasser $1,5 \times OPT$, sans jamais garantir d'atteindre l'optimum.

Q45. Le génétique donne-t-il l'optimum ?

Courte. Non, et lui n'offre même aucune garantie.

Q46. Qu'est-ce qu'un algorithme génétique ?

Courte. Une métaheuristique qui fait évoluer une population de solutions candidates par sélection, croisement et mutation.

Q47. Qu'est-ce qu'un individu chez vous ?

Courte. Une tournée complète, représentée par une permutation des 19 villes autres que Paris.

Développée. Le sujet parle joliment de « Théobald d'univers parallèles » : chaque individu est une version de Théobald qui aurait choisi un autre itinéraire.

Q48. Qu'est-ce que la fitness ?

Courte. La distance totale du cycle, retour au départ compris. Elle est à **minimiser**.

Q49. Pourquoi inclure le retour au départ ?

Courte. Parce que le sujet l'exige, et parce que sans lui l'algorithme optimiserait un chemin ouvert qui se terminerait loin de Paris.

Q50. Qu'est-ce qu'une génération ?

Courte. Une itération complète du cycle : évaluer, sélectionner, croiser, muter, remplacer. Nous en faisons 520.

Q51. Qu'est-ce que la sélection par tournoi ?

Courte. On tire k individus au hasard et on garde le meilleur comme parent. Chez nous $k = 5$.

Q52. Qu'est-ce que l'élitisme ?

Courte. Conserver les meilleurs individus tels quels d'une génération à la suivante. Chez nous, les 6 meilleurs sur 160.

Q53. Qu'est-ce qu'un seed ?

Courte. La graine du générateur pseudo-aléatoire. À graine identique, la suite de nombres tirés est identique, donc l'exécution est reproductible.

Q54. Pourquoi tester plusieurs seeds ?

Courte. Parce qu'une seule exécution d'un algorithme stochastique ne dit rien sur sa fiabilité.

Développée. Chez nous, la configuration « rapide » donne 3 334,21 km au seed 11 mais 3 157,13 km aux seeds 22 et 33. Avec un seul seed, on aurait conclu n'importe quoi.

Q55. Qu'est-ce qu'un écart-type ?

Courte. Une mesure de dispersion : la racine de la moyenne des carrés des écarts à la moyenne. $\sigma = 0$ signifie que toutes les valeurs sont identiques.

Q56. Qu'est-ce qu'un optimum local ?

Courte. Une solution meilleure que toutes celles de son voisinage immédiat, mais pas forcément la

meilleure de l'espace entier. **Développée.** Chez nous, 3 334,21 km est clairement un optimum local : deux configurations différentes s'y bloquent sur certains seeds.

Q57. Votre courbe de convergence prouve-t-elle que vous avez l'optimum ?

Courte. Non. Une courbe plate signifie que l'algorithme ne progresse plus, pas qu'il a trouvé le meilleur.

⚠ C'est la confusion la plus classique. Ne la faites pas.

Q58. Qu'est-ce qu'une borne inférieure ? Quelle est la vôtre ?

Courte. Une valeur dont on est sûr que l'optimum ne descend pas en dessous. La nôtre est le poids du MST : 2 665,05 km.

Q59. Qu'est-ce qu'une borne supérieure ? Quelle est la vôtre ?

Courte. Une valeur dont on est sûr que l'optimum ne dépasse pas. N'importe quelle tournée valide : la meilleure que nous ayons trouvée, 3 157,13 km.

Q60. Quel est votre encadrement de l'optimum ?

Courte. $2\,665,05 \leq \text{OPT} \leq 3\,157,13$, soit une fenêtre de 492,07 km. **Développée.** On peut en déduire que notre GA est au plus 18,5 % au-dessus de l'optimum et notre Christofides au plus 29,3 % — bien mieux que la garantie théorique de 50 %.

NIVEAU 3 — Questions techniques

Q61. Comment Prim fonctionne-t-il ?

Courte. On part d'un sommet, et à chaque étape on ajoute l'arête de poids minimal qui relie l'arbre à un sommet pas encore atteint, jusqu'à les avoir tous. **Développée.** C'est un algorithme glouton. On maintient les arêtes candidates dans un tas binaire (`heapq`), on dépile la moins chère, on vérifie que son extrémité n'est pas déjà visitée (sinon on la jette : elle créerait un cycle), on l'accepte, et on empile les nouvelles arêtes accessibles.

Q62. Quelle structure de données utilisez-vous dans Prim ?

Courte. Un tas binaire, via le module `heapq` de la bibliothèque standard. **Développée.** On y pousse des tuples (`poids`, `u`, `v`) ; `heapq` compare les tuples élément par élément, donc le tri se fait sur le poids. On utilise la suppression paresseuse : on ne retire pas les arêtes périmées du tas, on les ignore à la sortie.

Q63. Quelle est la complexité de votre Prim ?

Courte. $O(E \log V)$, soit $O(n^2 \log n)$ sur un graphe complet. **Développée.** Environ 820 opérations de tas pour 20 villes. Sur un graphe complet, un Prim naïf en $O(V^2)$ serait asymptotiquement meilleur, mais à cette taille c'est indiscernable, et la version avec tas reste efficace sur les graphes creux.

Q64. Prim ou Kruskal ?

Courte. Les deux donnent un MST de poids identique. Nous avons pris Prim : le graphe est dense, Prim n'a besoin que d'un tas alors que Kruskal demande en plus une structure union-find, et le sujet cite Prim dans sa base de connaissances. ⚠ Ne prétendez pas que Prim est « meilleur ». Assumez : Kruskal aurait convenu.

Q65. Prim ou Dijkstra ?

Courte. Ils résolvent des problèmes différents. Dijkstra minimise la distance depuis un sommet source ; Prim minimise le coût total de connexion du réseau. **Développée.** L'arbre des plus courts chemins depuis Paris relierait chaque ville directement à Paris ; le MST peut relier Nice à Marseille, ce qui coûte bien moins cher au total. Les deux arbres sont généralement différents.

Q66. Le sommet de départ de Prim change-t-il le poids du MST ?

Courte. Non. Nous l'avons vérifié pour les 20 sommets de départ : toujours 2 665,052352 km.

Développée. Quand les poids sont tous distincts — ce qui est le cas avec des distances géodésiques réelles — le MST est unique. Nous avons aussi vérifié que ce poids coïncide avec celui de `networkx.minimum_spanning_tree`.

Q67. Comment calculez-vous le couplage ?

Courte. Nous construisons le sous-graphe complet induit par les 12 sommets impairs — 66 arêtes — et nous appelons `networkx.algorithms.matching.min_weight_matching`.

Q68. Pourquoi ne pas avoir implémenté le couplage vous-mêmes ?

Courte. Parce que c'est l'algorithme des fleurs d'Edmonds, en $O(n^3)$: une brique d'optimisation combinatoire à part entière dont la réimplémentation n'aurait rien appris sur le TSP. **Développée.** Le sujet porte sur la modélisation et la comparaison de deux approches, et il cite NetworkX dans sa base de connaissances. Nous avons en revanche implémenté nous-mêmes Prim, la détection des degrés, l'union en multigraphe, Hierholzer et le shortcutting. Nous n'avons pas utilisé `networkx.approximation.christofides`.

Q69. `min_weight_matching` renvoie-t-il bien un couplage parfait ?

Courte. Oui dans notre cas. La fonction calcule un couplage de cardinalité maximale et de poids minimal ; sur un graphe complet à nombre pair de sommets, cardinalité maximale équivaut à couplage parfait. **Développée.** Nous l'avons vérifié : le résultat contient 6 arêtes et couvre exactement les 12 sommets impairs.

Q70. Comment gérez-vous les arêtes parallèles ?

Courte. Par des identifiants d'arête. Chaque arête du multigraphe reçoit un `edge_id` unique, et Hierholzer marque les arêtes consommées dans un ensemble d'identifiants. **Développée.** C'est justement pour ça qu'on n'utilise pas `nx.Graph` à cette étape : `nx.Graph` fusionnerait les arêtes parallèles, ce qui casserait la parité des degrés.

Q71. Votre Hierholzer est-il récursif ?

Courte. Non, itératif, avec une pile explicite. Pas de risque de dépassement de pile et c'est plus facile à suivre.

Q72. Combien d'arêtes dans votre multigraphe eulérien ?

Courte. 25 : les 19 du MST plus les 6 du couplage.

Q73. Combien pèse le circuit eulérien avant raccourcis ?

Courte. $2\,665,05 + 1\,142,03 = 3\,807,08$ km. **Développée.** Après raccourcis, la tournée ne fait plus que 3 445,60 km : les raccourcis ont fait gagner 361,48 km. C'est une vérification numérique directe de l'effet de l'inégalité triangulaire.

Q74. Christofides est-il déterministe chez vous ?

Courte. Oui. Aucun tirage aléatoire : l'ordre d'exploration de Hierholzer est entièrement déterminé par l'ordre de construction des listes d'adjacence.

Q75. Pourquoi OX et pas un croisement classique ?

Courte. Parce qu'un croisement classique produit des doublons et des villes manquantes : l'enfant ne serait plus une permutation, donc plus une tournée valide. **Développée.** OX conserve un segment du parent 1 et complète avec les villes manquantes dans l'ordre du parent 2. La validité est garantie par construction, il n'y a jamais besoin de réparer l'enfant.

Q76. Pouvez-vous dérouler un exemple d'OX ?

Courte. Oui.

```
Parent 1 : B C [D E F] G H
Parent 2 : D G B H C F E
On copie le segment D E F.
On lit le parent 2 : D(déjà), G, B, H, C, F(déjà), E(déjà) → il reste G, B, H, C
On remplit les trous de gauche à droite :
Enfant   : G B [D E F] H C
```

Chaque ville une seule fois : permutation valide.

Q77. Quelles mutations utilisez-vous ?

Courte. Deux, tirées à 50/50 : l'échange de deux villes (swap) et l'inversion d'un segment.

Développée. L'inversion est particulièrement intéressante : elle ne modifie que 2 arêtes de la tournée, alors que le swap en modifie 4. C'est exactement le mouvement de base de l'algorithme 2-opt, connu pour décroiser efficacement les arêtes.

Q78. Pourquoi les mutations préservent-elles la permutation ?

Courte. Parce qu'elles ne font que **déplacer** des gènes : rien n'est ajouté, rien n'est supprimé.

Q79. Pourquoi le tournoi plutôt que la roulette ?

Courte. Parce que le tournoi ne compare que des ordres, pas des valeurs : il ne nécessite ni fitness positive à maximiser ni mise à l'échelle. **Développée.** Nos distances sont toutes entre ~3 100 et ~6 500 km. Une roulette sur $1/\text{distance}$ donnerait des probabilités quasi identiques pour tous, donc une pression de sélection ridicule. Il faudrait ajouter un scaling, c'est-à-dire un paramètre de plus.

Q80. Que se passe-t-il si k est trop grand ? Trop petit ?

Courte. Trop grand : la pression est trop forte, seuls les meilleurs se reproduisent, la population s'uniformise et se bloque. Trop petit : la sélection devient quasi aléatoire et l'algorithme ne progresse plus. À $k = 1$, c'est du hasard pur.

Q81. Pourquoi 3 configurations et pas une seule ?

Courte. Parce que le sujet demande explicitement de « tester différentes configurations pour observer comment elles affectent la qualité des solutions ».

Q82. Comment choisissez-vous la configuration retenue ?

Courte. Le programme la choisit par un critère explicite : moyenne la plus faible, puis écart-type, puis meilleure valeur. **Développée.** C'est un tri lexicographique sur un triplet. L'équilibrée gagne dès le premier critère : 3 157,13 de moyenne contre 3 216,15 pour les deux autres. Le choix n'est donc pas fait à la main.

Q83. Pourquoi la configuration exploratoire n'est-elle pas meilleure ?

Courte. Parce que plus de calcul ne garantit pas un meilleur résultat : elle se bloque sur un optimum local au seed 22, malgré 800 générations et 240 individus. **Développée.** Et il y a une raison de conception : notre configuration « exploratoire » augmente la mutation, ce qui pousse à l'exploration, **mais aussi** la taille du tournoi, ce qui pousse à l'exploitation. Les deux réglages tirent en sens opposés. C'est une limite de notre plan d'expérience, que nous assumons.

Q84. Pourquoi « rapide » et « exploratoire » ont-elles exactement les mêmes chiffres ?

Courte. Parce qu'elles tombent sur les deux mêmes attracteurs — 3 157,13 et 3 334,21 km — simplement sur des seeds différents. **Développée.** La rapide échoue au seed 11 ; l'exploratoire échoue au seed 22. Dans les deux cas le multiset des trois résultats est {3 157,13 ; 3 157,13 ; 3 334,21}, donc toutes les statistiques coïncident. Ce n'est pas un bug : c'est le signe que notre instance possède deux optima locaux très marqués.

Q85. Pourquoi 520 générations ?

Courte. C'est la valeur de la configuration équilibrée, choisie empiriquement par le benchmark.

Développée. Et c'est en réalité largement trop : la meilleure tournée est atteinte dès la génération 67. Les 453 générations suivantes n'apportent rien. Un critère d'arrêt anticipé diviserait le temps de calcul par trois sans rien perdre.

Q86. Pourquoi une population de 160 ?

Courte. Empiriquement, c'est la taille de la configuration la plus stable sur nos seeds. Plus grand n'a pas amélioré le résultat, et coûte linéairement plus cher.

Q87. Pourquoi 22 % de mutation ?

Courte. Valeur empirique de la configuration retenue. **Développée.** Attention à la convention : chez nous, le taux s'applique à l'**enfant entier**, pas gène par gène. Les taux de 1 à 5 % qu'on lit dans la littérature s'appliquent souvent par gène, ce qui n'est pas comparable. ⚠ Ne dites jamais « 22 % est le taux optimal ».

Q88. Pourquoi 6 individus d'élite ?

Courte. 6 sur 160, soit 3,75 % — assez pour garantir la non-régression, assez peu pour ne pas uniformiser la population.

Q89. Pourquoi Paris est-il fixé dans le chromosome ?

Courte. Parce qu'un cycle est invariant par décalage : les 20 écritures du même cycle sont la même tournée. Fixer une ville supprime ces redondances sans perdre aucune solution. **Développée.** L'espace de représentation passe de 20! à 19!. Paris est simplement la première ligne du CSV, donc l'indice 0 ; le choix de la ville est arbitraire.

Q90. Cela oblige-t-il Théobald à partir de Paris ?

Courte. Non. Le cycle est le même quel que soit le point d'entrée : s'il habitait Lyon, il parcourrait exactement le même circuit avec la même distance, en le commençant à Lyon.

Q91. Quelle est la complexité de votre algorithme génétique ?

Courte. $O(G \times P \times n)$ pour les évaluations de fitness, plus $O(n^2)$ par croisement. **Développée.** Concrètement : $520 \times 160 = 83\,200$ évaluations, chacune sommant 20 distances, soit ~1,66 million d'opérations ; plus environ 40 000 croisements en $O(19^2)$. C'est ce qui explique l'écart de temps avec Christofides.

Q92. Le coût du GA dépend-il de la difficulté de l'instance ?

Courte. Non — uniquement de $G \times P$. Il fera 83 200 évaluations que l'instance soit facile ou difficile. **Développée.** C'est une différence fondamentale avec un algorithme exact, dont le coût explose avec la difficulté. Avantage : on choisit son budget. Inconvénient : la qualité n'est jamais garantie.

Q93. Pourquoi précalculer la matrice des distances ?

Courte. Parce que le GA fait environ 1,66 million de lectures de distance. Recalculer Haversine, avec ses sinus et cosinus, à chaque fois serait rédhibitoire. **Développée.** On ne calcule que le triangle supérieur — 190 appels au lieu de 400 — puis on miroite. La symétrie est ainsi garantie par construction.

Q94. Pourquoi Folium et Matplotlib ?

Courte. Folium produit une carte HTML interactive autonome, ouvrable dans n'importe quel navigateur — c'est la meilleure réponse à « affichez l'itinéraire sur la carte ». Matplotlib produit des images statiques pour les documents et la courbe de convergence.

Q95. Pourquoi JSON et CSV en sortie ?

Courte. Pour que les résultats soient exploitables sans relancer le calcul, à la fois par un humain et par un programme. Le JSON porte la structure complète, le CSV le tableau de benchmark.

NIVEAU 3 bis — Le professeur ouvre GitHub

Q96. Pourquoi une `dataclass` pour `City` ?

Courte. Elle génère `__init__`, `__repr__` et `__eq__`, et donne des attributs typés : on écrit `city.latitude` au lieu de `city[1]`. **Développée.** Elle est `frozen=True`, donc immuable et hashable : une ville ne doit jamais être modifiée en cours de calcul. Si un bug essayait de le faire, Python lèverait une exception au lieu de corrompre silencieusement les distances.

Q97. Pourquoi `frozen=True` sur `GAConfig` ?

Courte. Pour qu'aucun paramètre ne puisse être modifié au milieu d'une exécution, ce qui ruinerait la reproductibilité. **Développée.** C'est aussi pour ça qu'on reconstruit une config plutôt que de la muter : `GAConfig(**{**asdict(chosen), "seed": seed})`.

Q98. Pourquoi `ChristofidesResult` n'est-il pas `frozen` ?

Courte. C'est un simple conteneur de résultats renvoyé une fois, pas une valeur partagée qu'on risque de muter par accident. L'immutabilité n'y apporterait rien.

Q99. Pourquoi `ChristofidesResult` renvoie-t-il autant de champs ?

Courte. Parce que le sujet demande d'expliquer les étapes. En exposant le poids du MST, celui du couplage et la liste des sommets impairs, on peut présenter et vérifier chaque étape intermédiaire au lieu de sortir un nombre magique.

Q100. Pourquoi `encoding="utf-8-sig"` ?

Courte. Pour gérer le BOM que Windows et Excel ajoutent en tête des CSV. **Développée.** Sans ça, la première colonne s'appellerait " `Ville`" et notre validation de colonnes échouerait. C'est un piège très concret.

Q101. Pourquoi `newline=""` à l'ouverture du CSV ?

Courte. C'est la recommandation officielle du module `csv` : elle laisse le module gérer lui-même les fins de ligne, ce qui évite les lignes vides parasites sous Windows.

Q102. Pourquoi valider les colonnes avec `!=` et pas `issubset` ?

Courte. Pour refuser d'emblée un fichier mal formé, plutôt que de planter cinquante lignes plus loin avec un message incompréhensible. On veut une erreur claire et immédiate.

Q103. Pourquoi `if len(cities) < 3` ?

Courte. Parce qu'en dessous de 3 villes, la notion de cycle TSP n'a pas de sens : avec 2 villes il n'y a qu'un aller-retour.

Q104. Pourquoi `range(i + 1, n)` dans `distance_matrix` ?

Courte. Pour ne calculer chaque paire qu'une seule fois — 190 appels au lieu de 400 — puis miroiter avec `matrix[j][i] = d`. La symétrie est ainsi garantie par construction.

Q105. Pourquoi `range(1, len(cities))` dans `genetic_tsp` ?

Courte. Parce qu'on exclut l'indice 0, c'est-à-dire Paris, qui sert d'ancre du cycle. Le chromosome contient 19 gènes, pas 20.

Q106. Pourquoi `[0] + individual + [0]` ?

Courte. Pour reconstruire le cycle complet : on part de Paris, on suit la permutation, et on revient à Paris. C'est ce qui permet à `route_distance` de compter les 20 trajets.

Q107. Pourquoi `route_distance` ne referme-t-elle pas le cycle elle-même ?

Courte. Choix assumé : elle somme ce qu'on lui donne, ce qui la rend utilisable aussi pour un chemin ouvert. C'est `_closed_distance` qui ajoute les deux 0.

Q108. Pourquoi tous ces `.copy()` ?

Courte. Parce que sans copie, deux entrées de la population pointeraient sur la **même liste** : muter l'une modifierait l'autre. C'est le bug classique des GA en Python. **Développée.** On copie l'élite (`population[idx].copy()`) et le meilleur individu (`population[ranked[0]].copy()`).

Q109. Pourquoi `random.Random(seed)` et pas `random.seed()` ?

Courte. Pour avoir un générateur **local**. On ne touche pas au générateur global de Python : le résultat ne dépend pas de ce qui a été tiré ailleurs dans le programme, et deux exécutions ne peuvent pas s'influencer.

Q110. À quoi sert `sorted` dans `ranked = sorted(range(len(population)), key=...)` ?

Courte. À trier les **indices** de la population par distance croissante. On trie des indices et non des individus pour pouvoir remonter à la fois à l'individu et à sa distance déjà calculée.

Q111. Pourquoi cette `lambda` `key=lambda idx: distances[idx]` ?

Courte. Parce qu'on trie des indices mais qu'on veut trier selon les distances correspondantes : la `lambda` fait le lien entre les deux listes.

Q112. Pourquoi `min` et pas `max` dans le tournoi ?

Courte. Parce que notre fitness est une **distance à minimiser** : le meilleur individu est celui de plus petite valeur.

Q113. Pourquoi un `set` pour `seen` dans le shortcutting ?

Courte. Parce que le test d'appartenance est en $O(1)$ dans un `set`, contre $O(n)$ dans une liste.

Q114. Pourquoi `child = [-1] * n` dans OX ?

Courte. `-1` ne peut jamais être un indice de ville valide : il sert de marqueur « case vide » sans ambiguïté.

Q115. `gene not in child` est en $O(n)$. N'est-ce pas inefficace ?

Courte. Si, ça rend `make_child` en $O(n^2)$. Pour $n = 19$, c'est 361 opérations, donc négligeable — et c'est nettement plus lisible qu'un `set`. **Développée.** Sur une instance de plusieurs centaines de villes, il faudrait passer à un ensemble. C'est une optimisation identifiée, pas un oubli.

Q116. Pourquoi deux enfants par croisement ?

Courte. `make_child(a,b)` et `make_child(b,a)` exploitent mieux le couple de parents et remplissent la population deux fois plus vite.

Q117. Dans `sorted(rng.sample(range(n), 2))`, le dernier gène peut-il être dans le segment ?

Courte. Non. `right` vaut au maximum $n-1$ et le slice `child[left:right]` exclut l'indice `right`. Le tout dernier gène n'est donc jamais dans le segment conservé. **Développée.** C'est un biais mineur de l'implémentation, sans effet mesuré sur nos résultats — mais nous le connaissons et nous l'assumons.

Q118. Pourquoi `_mutate` ne retourne-t-elle rien ?

Courte. Elle modifie l'individu **en place**. C'est une convention Python classique (comme `list.sort()`) : une fonction qui mute ne retourne pas.

Q119. Pourquoi `pstdev` et pas `stdev` ?

Courte. `pstdev` est l'écart-type de **population** (division par n), `stdev` celui d'**échantillon** (division par $n-1$). Nous traitons nos 3 seeds comme la population complète de nos essais. **Développée.** Avec 3 valeurs, le choix change le chiffre — 83,48 contre 102,24 — mais pas le classement des configurations. C'est une convention que nous assumons, pas un oubli.

Q120. Pourquoi les garde-fous dans `genetic_tsp` ?

Courte. `elite_size >= population_size` rendrait la boucle de reproduction inutile et l'algorithme resterait figé. `population_size < 4` empêche un tournoi ou un croisement sensé.

Q121. Pourquoi `time.perf_counter()` et pas `time.time()` ?

Courte. `perf_counter` est l'horloge monotone de plus haute résolution, conçue pour mesurer des durées. `time.time()` peut sauter si l'horloge système est ajustée.

Q122. Pourquoi `GAConfig(**{**asdict(chosen), "seed": seed})` ?

Courte. La config est gelée : on ne peut pas modifier son seed. On la décompose en dictionnaire, on écrase uniquement le seed, et on reconstruit une nouvelle config. **Développée.** `dataclasses.replace(chosen, seed=seed)` ferait la même chose plus lisiblement — c'est une amélioration de style possible.

Q123. Que garantissent vos tests, exactement ?

Courte. Que les deux algorithmes renvoient un cycle **structurellement valide** : retour au départ, 21 éléments, exactement les 20 villes, aucun doublon, distance > 0 . **Développée.** Ils ne garantissent **pas** que la distance soit bonne, ni la valeur du MST, ni la correction de Haversine. Il n'y a pas non plus de test de non-régression sur les valeurs. C'est la première chose que nous ajouterions.

Q124. Pourquoi `unittest` et pas `pytest` ?

Courte. `unittest` est dans la bibliothèque standard : aucune dépendance supplémentaire à installer pour qui clone le dépôt. Pour deux tests structurels, c'est suffisant.

Q125. Pourquoi `_` devant certaines fonctions ?

Courte. Convention Python : elles sont **privées** au module, pas destinées à être appelées de l'extérieur. C'est le cas de `_tournament`, `_ordered_crossover`, `_mutate`, `_closed_distance` et `_hierholzer_multigraph`.

Q126. Le fichier `algogenetiques.py` à la racine, c'est quoi ?

Courte. Un brouillon de cours non fonctionnel, qui n'a jamais fait partie du projet. Il n'est pas suivi par Git et il ne s'exécute pas. ⚠ Si ce fichier est encore présent le jour de la soutenance, **supprimez-le** : il contient des erreurs de syntaxe et ne peut que semer la confusion. Le vrai algorithme génétique est dans `src/genetic.py`.

Q127. Vos résultats sont-ils reproductibles ?

Courte. Oui pour les distances : le seed est fixé, et nous avons reproduit exactement 3 445,60 km et 3 157,13 km à chaque exécution. **Développée.** Les **temps**, eux, varient avec la machine : nous avons mesuré 0,68 s puis 2,95 s pour la même exécution du GA sur deux machines. C'est normal et attendu.

NIVEAU 4 — Questions pièges

Q128. « Donc 3 157 km, c'est l'optimum ? »

Courte. Non. C'est la meilleure tournée que nous ayons observée. Nous n'avons pas calculé l'optimum exact. **Développée.** Ce que nous savons, c'est que l'optimum est entre 2 665,05 km, le poids du MST, et 3 157,13 km, notre meilleure tournée valide. $\triangle$ C'est le piège numéro un de tout l'oral.

Q129. « Christofides fait 1,5 fois l'optimum, donc l'optimum vaut 2 297 km ? »

Courte. Non. $3\,445,60 / 1,5 = 2\,297,07$ km est seulement une borne **inférieure** déduite de la garantie, pas une égalité. **Développée.** Et elle est moins bonne que celle du MST, qui donne $OPT \geq 2\,665,05$ km. Donc la garantie de Christofides ne nous apprend rien de plus ici : c'est logique, elle est conçue pour le pire cas et notre instance en est loin.

Q130. « Votre GA a convergé, donc il a trouvé le meilleur ? »

Courte. Non. Convergence signifie qu'il ne progresse plus, pas qu'il a atteint l'optimum global. Il peut être bloqué dans un optimum local. **Développée.** Nos seeds défavorables convergent à 3 334,21 km. Leur courbe est tout aussi plate, et pourtant ils sont 177 km au-dessus de notre meilleur résultat.

Q131. « Les algorithmes génétiques sont donc meilleurs que Christofides ? »

Courte. Sur cette instance et avec nos réglages, oui de 8,37 %. En général, non : ce sont deux outils différents. **Développée.** Christofides a une garantie, le génétique n'en a aucune. Nos propres seeds défavorables donnent 3 334,21 km, c'est-à-dire **moins bien** que Christofides. Et le génétique est $\sim 1\,000$ fois plus lent.

Q132. « Un autre seed pourrait-il donner un résultat pire ? »

Courte. Oui, absolument, et c'est arrivé : 3 334,21 km sur certains seeds avec les configurations rapide et exploratoire. **Développée.** C'est précisément pour ça que nous avons publié la moyenne, le pire et l'écart-type, et pas seulement le meilleur.

Q133. « Pourquoi ne pas garder le meilleur des 9 exécutions du benchmark ? »

Courte. Parce que ce serait mélanger deux choses : le benchmark sert à **choisir une configuration**, pas à produire le résultat final. **Développée.** Une fois la configuration choisie sur des critères de stabilité, on la relance sur les 3 seeds et on garde le meilleur. Le résultat publié correspond donc à une configuration identifiée, ce qui est reproductible.

Q134. « Vous gardez le meilleur de 3 exécutions : n'est-ce pas biaisé ? »

Courte. C'est la pratique standard avec une métaheuristique — on veut la meilleure tournée pour Théobald — mais il faut le dire honnêtement. **Développée.** Ce résultat ne caractérise pas la performance **typique** de l'algorithme. C'est exactement le rôle du benchmark, qui publie moyenne, pire et écart-type. Nous publions les deux.

Q135. « Votre écart-type de 0 prouve que votre algorithme est déterministe ? »

Courte. Non. Il montre que nos 3 seeds ont convergé au même endroit. L'algorithme reste parfaitement stochastique. **Développée.** Avec 30 seeds, on verrait probablement de la variance. Trois observations ne permettent aucune affirmation statistique.

Q136. « Pourquoi 3 seeds, pas 30 ? »

Courte. Par contrainte de temps de calcul. Nous assumons que c'est une limite réelle de notre étude. **Développée.** Le benchmark complet prend déjà 16 s. Avec 30 seeds il en prendrait ~ 160 s, ce qui reste faisable — c'est donc une amélioration prioritaire, et nous la listons comme telle.

Q137. « Plus de générations donne-t-il forcément un meilleur résultat ? »

Courte. Non. Notre configuration exploratoire fait 800 générations avec 240 individus et n'est pas meilleure que l'équilibrée avec 520 et 160. **Développée.** Et sur la configuration retenue, le meilleur résultat est atteint dès la génération 67 : les 453 suivantes ne servent à rien.

Q138. « Le MST fait 2 665 km : Théobald peut-il suivre ce trajet ? »

Courte. Non. Le MST n'est pas un circuit : il n'a pas de cycle, des villes y ont un seul voisin et d'autres en ont trois. Ce n'est pas un itinéraire. ⚠ Piège fréquent. Le MST est une **structure de connexion**, et son poids sert de borne inférieure.

Q139. « Vous utilisez NetworkX : vous n'avez donc pas implémenté Christofides ? »

Courte. Nous avons implémenté Prim, la détection des degrés impairs, l'union en multigraphe, Hierholzer et le shortcutting. NetworkX n'intervient que pour le couplage. **Développée.** Nous n'avons **pas** utilisé `networkx.approximation.christofides`, qui existe pourtant. Le couplage est l'algorithme des fleurs d'Edmonds, une brique d'optimisation combinatoire à part entière dont la réimplémentation n'aurait rien appris sur le TSP.

Q140. « Pourquoi vos temps ne correspondent pas à ceux du README ? »

Courte. Parce que ce sont des mesures expérimentales qui dépendent de la machine et de sa charge. **Développée.** Les **distances**, elles, sont identiques au chiffre près : 3 445,604687 et 3 157,127206. C'est ce qui compte, et c'est garanti par le seed fixé.

Q141. « Haversine donne la vraie distance que parcourra Théobald ? »

Courte. Non, c'est la distance à vol d'oiseau. Le trajet routier réel est typiquement 15 à 30 % plus long. **Développée.** Paris-Marseille : 661 km à vol d'oiseau, environ 775 km par la route. Mais ce biais s'applique à **toutes** les tournées, donc la comparaison entre les deux algorithmes reste valable — et c'est elle qui fait l'objet du sujet.

Q142. « Vos 20 villes ne sont pas des marchés médiévaux. »

Courte. Exact : ce sont les 20 plus grandes villes françaises actuelles, fournies avec le sujet. L'habillage médiéval est narratif ; le problème mathématique est le même.

Q143. « Le TSP est NP-difficile : comment votre code trouve-t-il une solution en 1,5 ms ? »

Courte. Parce qu'il ne trouve pas **la** solution optimale, mais **une** bonne solution. Trouver une tournée valide est trivial dans un graphe complet ; c'est trouver la plus courte qui est difficile.

Q144. « Y a-t-il un cycle hamiltonien dans votre graphe ? »

Courte. Oui, et même $19!/2$ cycles distincts : dans un graphe complet, **toute** permutation en est un. C'est trouver le plus court qui est difficile.

Q145. « Si vous inversez le sens de la tournée, la distance change-t-elle ? »

Courte. Non, le graphe est non orienté. C'est d'ailleurs une symétrie que nous n'avons pas éliminée de la représentation du GA — une amélioration possible.

Q146. « Comment prouvez-vous que votre tournée est valide ? »

Courte. Par nos tests unitaires : ils vérifient que la route revient au départ, qu'elle a 21 éléments, qu'elle contient exactement les 20 villes et aucun doublon. **Développée.** Et visuellement sur les cartes : chaque marqueur porte son numéro d'ordre, de 1 à 20, sans répétition.

Q147. « P = NP a-t-il été résolu ? »

Courte. Non, c'est un problème ouvert, l'un des sept problèmes du millénaire. $\triangle$ Ne dites jamais « on a prouvé qu'il n'existe pas d'algorithme rapide ». On ne l'a pas prouvé.

Q148. « Quelle est la plus grosse faiblesse de votre étude ? »

Courte. Le benchmark à 3 seeds. Il suffit à départager nos configurations de façon argumentée, mais pas à faire une affirmation statistique. **Développée.** Juste derrière : le fait que nos trois configurations font varier plusieurs paramètres à la fois, ce qui empêche d'isoler l'effet de chacun. $\triangle$ Ne répondez jamais « aucune ». Un professeur qui pose cette question teste votre lucidité.

Q149. « Qu'est-ce que "robuste" veut dire exactement ici ? »

Courte. La capacité à donner un résultat de qualité comparable d'une exécution à l'autre. On la mesure par l'écart-type sur plusieurs seeds. **Développée.** Christofides est robuste par construction : il est déterministe. Notre configuration retenue a $\sigma = 0$ km sur 3 seeds, ce qui est excellent, mais mesuré sur trop peu d'observations pour être une preuve.

Q150. « Si je vous donne 1 000 villes, que faites-vous ? »

Courte. Christofides, sans hésiter. Il reste polynomial et garanti ; notre GA, avec ces paramètres, serait à la fois trop lent et bien trop loin de l'optimum. **Développée.** Il faudrait vérifier le coût du couplage, en $O(k^3)$ sur les sommets impairs — potentiellement 500 sommets, donc $\sim 10^8$ opérations. Et en pratique, on enchaînerait avec du 2-opt ou du Lin-Kernighan.

NIVEAU 5 — Questions très exigeantes

Q151. « Démontrez que le couplage coûte au plus OPT/2. »

Réponse. Soit O l'ensemble des sommets impairs, $|O| = 2k$ (pair par le lemme des poignées de main). Parcourez la tournée optimale T^* et ne retenez que les sommets de O , dans l'ordre de rencontre : vous obtenez un cycle C sur O . Chaque arête de C court-circuite un morceau de T^* ; par inégalité triangulaire, elle coûte au plus ce morceau. Donc $\text{poids}(C) \leq \text{OPT}$. Comme C a un nombre **pair** de sommets, on peut le décomposer en deux couplages parfaits disjoints A et B en prenant une arête sur deux. On a $\text{poids}(A) + \text{poids}(B) = \text{poids}(C) \leq \text{OPT}$, donc $\min(A, B) \leq \text{OPT}/2$. Comme notre couplage W est de poids **minimal** sur O , il est au moins aussi bon : $W \leq \min(A, B) \leq \text{OPT}/2$. ■ $\triangle$ L'étape qui demande la minimalité est la dernière. Sans elle, la preuve est fausse.

Q152. « La garantie de 1,5 tient-elle si les distances ne sont pas métriques ? »

Courte. Non. La preuve utilise l'inégalité triangulaire à deux endroits : dans le raccourci et dans la construction du cycle C sur les sommets impairs. **Développée.** Pour le TSP général non métrique, on sait qu'aucun algorithme d'approximation à facteur constant n'existe, sauf si $P = NP$.

Q153. « Le facteur 1,5 a-t-il été amélioré ? »

Courte. Oui, en 2020 : Karlin, Klein et Oveis Gharan ont publié un algorithme en $1,5 - \epsilon$ avec ϵ de l'ordre de 10^{-36} . **Développée.** C'est une percée théorique majeure — le 1,5 de Christofides tenait depuis 1976 — mais sans aucune portée pratique à cette valeur d'épsilon.

Q154. « Quelle est la meilleure borne inférieure connue pour le TSP ? »

Courte. La borne de Held-Karp, obtenue par relaxation lagrangienne du problème, atteint souvent 99 % de l'optimum. **Développée.** Nous utilisons le MST, bien plus faible (typiquement 10 à 25 % en dessous),

mais immédiat à calculer puisqu'on le construit déjà pour Christofides.

Q155. « Pourquoi ne pas avoir calculé l'optimum exact ? Held-Karp est en $O(n^2 \cdot 2^n)$. »

Courte. Pour $n = 20$: $400 \times 1\,048\,576 \approx 4,2 \times 10^8$ opérations et une table de plusieurs centaines de Mo en Python. C'est à la limite du faisable et hors du périmètre du sujet. **Développée.** Ce serait faisable à $n \leq 15$, et c'est précisément l'amélioration que nous proposons : calculer l'optimum exact sur une sous-instance pour calibrer l'écart réel de nos deux méthodes.

Q156. « Vos deux algorithmes sont-ils comparables sur le même budget de temps ? »

Courte. Non, et c'est une vraie limite de notre comparaison. Christofides prend 1,5 ms, le GA 1,2 s.

Développée. Une comparaison plus rigoureuse donnerait le même budget aux deux : à budget égal, on lancerait Christofides puis du 2-opt pendant le temps restant, et il n'est pas du tout évident que le GA gagnerait encore. $\triangle$ Excellente question. Assumez la limite plutôt que de la contourner.

Q157. « Vos trois configurations font varier plusieurs paramètres à la fois. »

Courte. Exact, c'est une faiblesse de notre plan d'expérience : on ne peut pas isoler l'effet d'un paramètre. **Développée.** Il faudrait un plan factoriel, ou faire varier un facteur à la fois. Et pour comparer les distributions, un test de Wilcoxon ou de Mann-Whitney serait nécessaire — ce que 3 observations ne permettent pas.

Q158. « Votre configuration "exploratoire" est-elle vraiment exploratoire ? »

Courte. Pas entièrement. Elle augmente la mutation, ce qui pousse à l'exploration, mais aussi la taille du tournoi, ce qui pousse à l'exploitation. **Développée.** Les deux réglages tirent en sens opposés, ce qui explique en partie qu'elle ne surpasse pas l'équilibrée malgré 3,6 fois plus de calcul. C'est un défaut de conception que nous avons identifié après coup.

Q159. « Peut-on combiner Christofides et l'algorithme génétique ? »

Courte. Oui, et ce serait notre amélioration la plus élégante : injecter la tournée Christofides comme individu de la population initiale du GA. **Développée.** On part alors de 3 445,60 km garantis, et comme l'élitisme empêche toute régression, le GA ne peut qu'améliorer. On obtient le meilleur des deux mondes : la garantie de Christofides devient un plancher de qualité. C'est ce qu'on appelle un algorithme **mémétique** — une métaheuristique de population couplée à une méthode de construction ou de recherche locale.

Q160. « Pourquoi vos deux optima locaux sont-ils à 3 157 et 3 334 km ? »

Courte. Nous ne l'avons pas analysé en détail, mais l'écart de 177 km suggère un ou deux croisements d'arêtes non résolus dans la variante à 3 334 km. **Développée.** C'est exactement le type de blocage qu'une passe de 2-opt éliminerait : elle décroise systématiquement les arêtes qui se chevauchent. C'est un argument fort pour notre amélioration prioritaire. $\triangle$ Si vous n'avez pas vérifié, dites-le. Ne fabriquez pas une analyse que vous n'avez pas faite.

Q161. « Comment mesureriez-vous la diversité de votre population ? »

Courte. Par exemple par la distance de Hamming moyenne entre chromosomes, ou par le nombre d'arêtes distinctes présentes dans la population. **Développée.** Nous ne l'avons pas fait. Ce serait pourtant un excellent indicateur : une chute brutale de diversité expliquerait la convergence prématurée observée sur les seeds défavorables.

Q162. « Pourquoi ne pas avoir utilisé ERX plutôt qu'OX ? »

Courte. ERX est réputé meilleur pour le TSP parce qu'il raisonne en **arêtes**, ce qui correspond exactement à ce qui coûte, alors qu'OX raisonne en positions. **Développée.** Nous avons choisi OX pour sa simplicité et parce qu'il garantit la validité sans réparation. Tester ERX est une amélioration légitime que nous assumons ne pas avoir faite.

Q163. « Votre GA respecte-t-il vraiment la métaphore biologique ? »

Courte. Non, et il n'en a pas besoin. L'élitisme rend certains individus immortels, ce qui n'a aucun sens biologique. **Développée.** Un algorithme génétique n'est pas une simulation : c'est une méthode de recherche stochastique guidée. La métaphore aide à comprendre, elle ne contraint pas la conception. Nous optimisons, nous ne simulons pas.

Q164. « Que se passerait-il avec un taux de croisement de 100 % ? »

Courte. Aucun parent ne passerait intact : toute la descendance serait recombinaisonnée. Ce n'est pas catastrophique — l'élitisme protège les meilleurs — mais on perd le mécanisme qui laisse une bonne solution se propager sans altération. **Développée.** À 95 %, environ un couple sur vingt passe tel quel, ce qui est un léger supplément d'exploitation.

Q165. « Votre matrice de distances est-elle exacte ? »

Courte. Elle est exacte au sens du modèle sphérique. Nous avons vérifié qu'elle est symétrique, de diagonale nulle, et qu'aucun des 6 840 triplets ne viole l'inégalité triangulaire. **Développée.** Ce que nous n'avons pas fait, c'est comparer une valeur à une référence externe connue. C'est un test que nous devrions ajouter — c'est la brique la plus fondamentale du projet.

Q166. « Combien d'évaluations de fitness votre GA fait-il ? »

Courte. 83 200 : 160 individus $\times$ 520 générations, pour une seule exécution. **Développée.** Multiplié par 20 additions chacune, cela fait $\sim 1,66$ million d'opérations pour la seule fitness, sans compter les croisements. C'est le poste de coût numéro un, et le plus facile à optimiser — via NumPy, ou via un calcul incrémental de la variation de distance après mutation.

Q167. « Pourriez-vous calculer la fitness de façon incrémentale ? »

Courte. Oui, pour les mutations. Une inversion de segment ne modifie que 2 arêtes : on pourrait mettre à jour la distance en $O(1)$ au lieu de la recalculer en $O(n)$. **Développée.** Pour un croisement OX, en revanche, la tournée change trop pour un calcul incrémental simple. Le gain porterait donc surtout sur les mutations.

Q168. « Si Théobald ne pouvait rouler que 300 km par jour, que changeriez-vous ? »

Courte. Le problème changerait de nature : ce ne serait plus un TSP mais un problème de tournées avec contraintes, proche du VRP. **Développée.** C'est justement là que l'algorithme génétique devient très supérieur : il suffirait de pénaliser la fitness des tournées violant la contrainte, ou de segmenter la tournée en étapes journalières. Christofides, lui, ne saurait pas quoi faire d'une telle contrainte — sa construction est rigide. $\triangle$ C'est le meilleur argument en faveur du GA. Gardez-le pour la fin.

Q169. « Et s'il y avait plusieurs marchands ? »

Courte. Ce serait un mTSP, ou un VRP s'il y a des capacités. Même réponse : le GA s'adapte en changeant la représentation et la fitness, Christofides ne s'applique plus directement.

Q170. « Votre projet serait-il différent avec 6 villes ? »

Courte. Avec 6 villes, $5!/2 = 60$ tournées distinctes : on calculerait l'optimum exact par force brute en quelques millisecondes. **Développée.** Et ce serait très instructif : on saurait enfin de combien Christofides et le GA s'écartent réellement de l'optimum. C'est exactement l'esprit de notre amélioration

« Held-Karp sur une sous-instance ».

Q171. « Pourquoi le génétique bat-il Christofides, concrètement, sur la carte ? »

Courte. Dans le sud-est. Christofides fait Nîmes → Nice → Toulon → Marseille, donc il monte à Nice puis redescend : deux arêtes se croisent. Le génétique fait Nîmes → Marseille → Toulon → Nice → Grenoble, sans croisement. **Développée.** C'est là que se jouent l'essentiel des 288 km. La construction de Christofides est rigide : elle suit l'ordre imposé par le circuit eulérien et ne peut pas revenir en arrière pour décroiser. C'est exactement ce que ferait un 2-opt. $\triangle$ C'est la réponse la plus impressionnante de toute la banque, parce qu'elle montre que vous avez **regardé** vos cartes, pas seulement vos chiffres.

Q172. « Qu'avez-vous appris ? »

Courte. Qu'une garantie théorique et une bonne performance pratique sont deux choses différentes, et qu'il faut mesurer plusieurs fois avant de conclure quoi que ce soit sur un algorithme stochastique.

Développée. Concrètement : Christofides nous garantit de ne jamais dépasser $1,5 \times \text{OPT}$, et pourtant il perd de 8 % face à une méthode sans aucune garantie. À l'inverse, cette même méthode sans garantie perd face à Christofides sur un tiers de nos seeds. Aucune des deux n'est « meilleure » dans l'absolu — c'est le contexte d'usage qui tranche.

Les 20 questions les plus probables

Si vous manquez de temps, révisez celles-ci en priorité :

#	Question
Q4	Pourquoi ne pas tout tester ?
Q5	Pourquoi $(n-1)!$?
Q9	Pourquoi 190 arêtes ?
Q13	Pourquoi Haversine ?
Q22	Pourquoi l'inégalité triangulaire ?
Q27	Pourquoi $\text{MST} \leq \text{OPT}$?
Q32	Pourquoi les sommets impairs sont-ils en nombre pair ?
Q35	Pourquoi un couplage <i>minimal</i> ?
Q38	Euler vs Hamilton ?
Q43	D'où vient le 1,5 ?
Q64	Prim ou Kruskal ?
Q65	Prim ou Dijkstra ?
Q75	Pourquoi OX ?
Q82	Comment choisissez-vous la configuration ?
Q89	Pourquoi Paris est-il fixé ?
Q128	Est-ce l'optimum ?
Q130	La convergence prouve-t-elle l'optimum ?
Q131	Le GA est-il meilleur que Christofides ?
Q148	Quelle est votre plus grosse faiblesse ?
Q168	Et avec une contrainte de 300 km/jour ?

Banque construite à partir du sujet officiel et de notre implémentation réelle. Les valeurs citées sont celles de `results/summary.json`.